TRADED RISK MODEL DEVELOPMENT · OCTOBER 2026

Context engineering for AI-assisted model development



What the agent needs to know, where it lives in VS Code, and how to keep the window clean, worked on a Black–Scholes pricer for equity options

VS Code · GitHub Copilot · Black–Scholes equity pricer

WHY CONTEXT

The agent only knows what is in its window

Each request is assembled from seven sources. Anything outside them does not exist for the model.

More is not better: in Chroma's 2025 test of 18 models, reliability fell as input grew.

The aim is the smallest set of high-signal tokens that makes the right output likely (Anthropic, 2025).

ONE REQUEST = ONE CONTEXT WINDOW

System instructions	built in
Customizations	you, once per repo
Your message	you, every request
Conversation history	grows every turn
Implicit context	active file, selection
Explicit references	you, via #file
Tool outputs	grows as the agent works

When it fills, VS Code compacts older turns into a summary, and detail can be lost.

RUNNING EXAMPLE

Adding a Black–Scholes pricer for equity options

Little code, many conventions: every step rests on one that the agent has to get exactly right.

Inputs
S, K, T, r, q, σ

CONVENTION
`day count`

→

Forward
$F = S \cdot \exp((r - q) \cdot T)$

CONVENTION
`dividends`

→

d1, d2
$d1 = \ln(F/K)/(\sigma\sqrt{T}) + \sigma\sqrt{T}/2$
$d2 = d1 - \sigma\sqrt{T}$

CONVENTION
`vol units`

→

Price
$C = DF \cdot [F \cdot N(d1) - K \cdot N(d2)]$
$DF = \exp(-r \cdot T)$

CONVENTION
`exercise style`

→

Greeks
Δ, Γ, vega, θ, ρ

CONVENTION
`Greek units`

WHAT WE TYPE

```
Add a Black-Scholes pricer for European equity
options in src/pricing/bs.py
```

WHAT IT MUST ALSO KNOW

Our day count · dividend model · vol and Greek units · exercise style · the market-data API · what counts as validated

WITHOUT CONTEXT

Five ways the agent gets the pricer wrong

What the agent does	Likely cause	Context that prevents it
Computes T with its own day count, not the one our vol surface uses	Several conventions are common	T only via pricing/dates.year_fraction()
Quotes vega per 1.00 of vol instead of per vol point: off by 100×	Units are invisible in the signature	Units stated in AGENTS.md; a bump-and-reprice test
Drops the dividend yield, or turns discrete dividends into a flat yield	Textbook Black–Scholes has no dividends	Dividend model stated in the scoped instructions
Prices American single-stock options with the European formula	The prompt said only “equity options”	Scope rule: European only; American goes to the tree pricer
Returns NaN, or breaks no-arbitrage bounds, as $T \to 0$ or $\sigma \to 0$	Edge cases are rare in tutorials	Benchmarks: Hull’s 42/40 case, put–call parity, the limits

WHERE IT LIVES

Each piece of knowledge has a home in VS Code

LAYER	FILE IN THE REPO	LOADS	IN OUR EXAMPLE
Always-on	`AGENTS.md` or `.github/copilot-instructions.md`	Every request	Stack, test command, units, data rules
Scoped	`.github/instructions/pricing.instructions.md`	applyTo match, or the task fits its description	Day count, dividends, Greek units, scope
Workflow	`.github/skills/explain-price-break/SKILL.md, waterfall.py, report.md`	A price break, or /explain-price-break	Checklist, waterfall script, report template
Role	`.github/agents/pricing-reviewer.agent.md`	You pick it, or another agent delegates	Read-only review against the model spec
Per request	`#file:src/marketdata/api.py` + the failing test output	You attach it	Exact inputs for this one task

Prompt files (.prompt.md) run only in the Local harness; for workflows you reuse across Copilot clients, write a skill.

IN THE REPO

Two short files carry most of the context

AGENTS.md every request

```
# Equity derivatives pricing library
Python 3.12, numpy, scipy, pytest. Run: make test
Units: rates, vols, yields as decimals (0.20)
T in years via pricing/dates.year_fraction()
Model specs: docs/model-spec/
Market data: only through src/marketdata/api.py
Test data: synthetic only, in tests/data/
Ask before changing src/pricing/api.py signatures
```

Left out on purpose

Style rules the linter already enforces, and anything the agent can read from the code itself.

.github/instructions/pricing.instructions.md on match

```
---
description: Rules for analytic option pricers
applyTo: "src/pricing/**,tests/**"
---
- European exercise only. American: route to the
  tree pricer and say so; never approximate.
- Dividends: continuous yield q from market data.
  Discrete dividends: stop and ask.
- Vega and rho per 1% move, theta per calendar
  day. Why: the risk system reports those units.
- T → 0 or σ → 0: return DF·max(F - K, 0) for a
  call (put alike); never NaN.
- Cite the model spec section in every docstring.
- Done = make test green, incl. tests/benchmark/.
```

REUSABLE WORKFLOW

A skill packages the workflow, loaded on demand

`.github/skills/explain-price-break/SKILL.md` on demand

```
---
name: explain-price-break
description: Explain why our price or Greek
  differs from another source (front office,
  vendor). Use for breaks and reconciliations.
---
1. Collect both input sets, with units, in
   [inputs.json](./inputs.json) format.
2. Normalise units; check day count, vol units,
   dividends, exercise style, T or σ near 0.
3. Run [waterfall.py](./scripts/waterfall.py) to
   swap inputs one at a time. No hand arithmetic.
4. Residual above tolerance: it is a model
   difference; say so and stop.
5. Write it up with [report.md](./report.md).
```

HOW IT LOADS

1 · Always in context

Name and description, one line per skill. The description must say when to use it.

2 · On a price break, or /explain-price-break

The SKILL.md body: five steps, one checklist

3 · Only when a step links to it

inputs.json, waterfall.py, report.md

Sample run on the bundled inputs.json: front office 12.44 vs our 11.12. Day count +0.08, dividends +1.23, residual +0.004: explained by inputs.

DURING THE TASK

Run each task in phases with a clean window

1 · Explore

`subagent, read-only`

Reads widely in its own window; only a summary comes back.

2 · Plan

`Plan agent → file`

Save plans/bs-pricer.md; check conventions, edge cases, scope.

3 · Implement

`new chat + plan file`

Benchmark tests first, then code, then make test.

4 · Review

`fresh chat, reviewer`

Checks the diff against the plan and model spec.

Throughout the task

Context meter

In the chat input. Compact before it fills.

`/compact`

Give it a focus: keep decisions, drop logs.

`/fork`

Try an alternative without losing the thread.

Checkpoints

Rewind file edits; terminal effects stay.

BEST PRACTICES AND PITFALLS

What works, and what to avoid

Do

Rules on evidence. Add one after the agent repeats a mistake, and give the reason.

Point, don't paste. Reference the model spec and APIs so the agent reads the source.

Executable "done". Tests and benchmark results the agent runs itself.

Short always-on files. Scope the rest with applyTo and skills.

Context is code. Version it and review it in pull requests.

Check what loaded. References in the reply; Agent Debug Logs.

Avoid

Context dumping. Whole PDFs or 50k-row files; give a schema and 20 rows.

Marathon chats. Failed attempts keep steering; compaction drops detail.

Conflicting files. Instruction sources add up; nothing takes precedence.

Recalled conventions. And shared skills nobody has reviewed.

Every tool switched on. Worse tool choice; fetched pages can inject instructions.

Sensitive data in prompts. No positions, client names or production paths.

NEXT WEEK

Start with three files and one habit

`AGENTS.md`

One page: stack, test command, units, data rules.

`pricing.instructions.md`

Conventions and scope, each with its reason.

`docs/model-spec/ + tests/benchmark/`

Ground truth the agent reads, and must pass.

The habit: one task per chat. Explore, plan to a file, implement in a fresh chat, and let the tests decide when it’s done.

REFERENCE

Sources and what is illustrative

Sources

VS Code documentation, accessed 24 Oct 2026: Context, Custom instructions, Prompt files, Agent skills, Subagents, Best practices, Context engineering guide.

Anthropic Engineering, Effective context engineering for AI agents (Sep 2025).

Chroma, Context Rot: How Increasing Input Tokens Impacts LLM Performance (2025).

F. Black and M. Scholes, The Pricing of Options and Corporate Liabilities, Journal of Political Economy (1973); R. C. Merton, Theory of Rational Option Pricing, Bell Journal of Economics and Management Science (1973), for the dividend yield.

J. C. Hull, Options, Futures, and Other Derivatives, 9th ed., Example 15.6: S = 42, K = 40, r = 10%, σ = 20%, T = 0.5 gives call 4.76 and put 0.81, recomputed for this deck.

Illustrative, not prescriptive

Repository layout, file and skill names, and make test.

The conventions: day count, Greek units, dividend model, and routing American options to a tree pricer.

The failure modes are expected behaviour, not logged incidents.